Universidad de Buenos Aires
Facultad de Ciencias Exactas y Naturales
Departamento de Computación

[your title here]

Tesis de Licenciatura en Ciencias de la Computación

Rafael Romani

Director: Santiago Cifuentes
Codirector: Santiago Figueira
Buenos Aires, 2026

[YOUR TITLE HERE]

[Acá va el resumen]

Palabras claves: [acá van las palabras clave]

[YOUR TITLE HERE]

[write your abstract here]

Keywords: [write your keywords here]

[acá van los agradecimientos]

CONTENTS

1. INTRODUCTION

1.1 Uso y poder de los LLMS

In 2017, a group of researchers working at Google published a paper introducing a technology that revolutionized society. In *Attention is All You Need*, they presented the transformers, today's core of large language models (LLMs) and generative AI.

This architecture was build intended as text sequence-to-sequence model for machine translation but today has applications in various fields as natural language processing, computer vision or speech recognition.

The first model implementing transformers open for public use was ChatGPT by OpenAI. It made its debut in November 2022 and by December 2024 it reported 300 million global weekly users. Now there are many models competing for this market as Claude, from Anthropic, or Gemini from Google.

These models can be all be grouped under the generative AI tools. This is a technology that is arguably the most rapid technology adopted by society. In the second half of 2025, roughly one in six people worldwide were using generative AI tools. not sure how to cite this https://www.microsoft.com/en-us/research/wp-content/uploads/2026/01/Microsoft-AI-Diffusion-Report-2025-H2.pdf

SF: Lo citás como misce-laneous

LLMs are being used not only as chatbots for asking small questions, but are shaping the industry and future. For example, [**?**] found that more than 17% of the Computer Science's papers published between January 2020 and February 2024 on the *arXiv*, *bioRxiv*, and *Nature* portfolio journals had sentences heavily edited by ChatGPT.

Missing usage in industry

So naturally, the question about their computing power pops up. Are this kind of models really capable of doing the stuff for what they are being used?

- well-suited to parallelism enables transformer models to take full advantage of the power and speed offered by GPUs during both training and inference. This possibility, in turn, unlocked the opportunity to train transformer models on unprecedentedly massive datasets through self-supervised learning.
- chatgpt, claude, gemini,
- every day users
- number of users?
- use in the industry?
- other uses besides the chatbots?

cite this https://www.ibm.co model

1.2 Pero qué es un LLM?

1.3 Formalizaciones previas y resultados que ignoran los aspectos probabilísticos

1.4 Objetivos y contribuciones

2. THEORETICAL FRAMEWORK

2.1 Transformers

In this work we are going to analyze transformers as language recognizers. A transformer is a machine that works over an alphabet Σ. It has only one tape from where it reads the input and in an autoregressive

SF no sé bien a qué se refiere con autoregressive, pero tiene que quedar claro que solo imprime nuevos símbolos de a uno y que no puede modificar símbolos ya escritos. Quizá comparar con autómata vs TM

way writes in it more tokens. There are two distinguished tokens called **decision symbols**. Once the transformer writes one of those, it halts. We say that a transformer accepts a word if, when it halts, the decision symbol printed is the accepting one (q_{accept}). Analogously, a transformer rejects a word if the decision symbol printed is the rejecting one (q_{reject}).

In each step, the transformer maps the content of the tape to a probability distribution over Σ. Depending on this distribution, the transformer prints the next token. The transformer iterates this process until it halts. These iterations are called **chain of thought**.

The process of generating the next token consists of two separated steps. First a probability distribution over the alphabet for the next token is computed by a process called distribution generation.

SF así como hiciste con cot, definí nuevos términos con un estilo especial. Yo uso defstlye como comando y después decido si es itálica, negrita, etc

After that, it is necessary to decide the specific token by a process called token selection. These two algorithms together is what defines the transformer. During this work we will analyze and compare two different options for token selection.

For a more realistic analysis of the transformers, we don't allow an infinite tape. The transformers studied in this work will have, for each input size n, a tape of size $\text{budget}(n) + n$. It's important to remark that, since in every step of the transformer a symbol is written, the maximum number of steps a transformer can do for an input of length n is $\text{budget}(n)$.

SF: de hecho, no habías dicho que era infinita, pero es importante remarcarlo

The process of distribution generation is dependent on many parameters. These parameters, in the real models, are the ones calculated during the training. We refer to them as θ. We consider the parameters as part of the definition of a transformer.

We note $\mathsf{TF}(\alpha)$ to the token outputted by one step of the transformer TF when the tape content is $\alpha \in \Sigma^*$. The autoregressive token generation is defined recursively. The token outputted after i steps is $\mathsf{TF}^i(\alpha) := \mathsf{TF}^{i-1}(\alpha\mathsf{TF}(\alpha))$ with base case $\mathsf{TF}^1(\alpha) := \mathsf{TF}(\alpha)$. In other words, the output of the ith step is $\sigma_i = \mathsf{TF}^i(\alpha) = \mathsf{TF}(\alpha\sigma_1 \dots \sigma_{i-1})$ where $\sigma_j \in \Sigma$ for all j.

SF: los pasos son del cómputo, nodel transformer. Revisar en todo el texto

We note $\mathsf{TF}^*(\alpha)$ for the token written by the transformer when it halts, i.e. at the end of the computation, when the input is α. As was said before, the transformer only halts when it prints a decision symbol (q_{accept} or q_{reject}). Therefore, $\mathsf{TF}^*(\alpha) \in \{q_{accept}, q_{reject}\}$.

As stated before, the **distribution generation** is a mapping of the content of the tape to a probability distribution over Σ. There are different algorithms for this process in the literature. All the transformers considered in this work use the one presented by [**?**]. This algorithm consists of four parts: a token embedding layer (TE), a position encoding layer (PE), an output linear layer (OUTPUT), and a stack of L identical decoder layers, where L is also called the depth of the model. Each decoder layer has two sub-layers: a multi-head self-attention layer (ATTN) and a position-wise fully-connected feed-forward network (FF). Each part and layer has its own trainable parameters. Then, we can define the distribution generation algorithm by its parameters $\theta = \{\theta_{\mathsf{TE}}, \theta_{\mathsf{PE}}, \{\theta^l_{\mathsf{ATTN}}, \theta^l_{\mathsf{FF}}\}_{l=0}^{L-1}, \theta_{\mathsf{OUTPUT}}\}$. The algorithm works over vectors of certain dimension d. Let's define each component.

Token Embedding: It's a mapping from Σ to vectors in $\mathbb{R}^d$. This mapping is defined by $\theta_{\mathsf{TE}} \in \mathbb{R}^{d\times|\Sigma|}$. For all $\sigma \in \Sigma$, we note $\theta_{\mathsf{TE}}(\sigma) \in \mathbb{R}^d$ to the mapping of token σ.

Position Encoding: It's a mapping from the positions of the tape to vectors in $\mathbb{R}^d$. This mapping is defined by $\theta_{\mathsf{PE}} \in \mathbb{R}^{d\times \text{budget}(n)+n}$. For all position of the tape $1 \leq i \leq \text{budget}(n) + n$ we note $\theta_{\mathsf{PE}}(i) \in \mathbb{R}^d$ to the mapping of position i.

Self Attention Layer: It's a mapping from $(\mathbb{R}^d)^a$ to $(\mathbb{R}^d)^a$. Given parameters $\theta_{\mathsf{ATTN}} = (W_Q, W_K, W_V, W_O) \in \mathbb{R}^{d\times d} \times \mathbb{R}^{d\times d} \times \mathbb{R}^{d\times d} \times \mathbb{R}^{d\times d}$, the layer it's defined by the algorithm 1. This layer is defined only as a single head attention layer because we can simulate 1 multi-head attention layer with any constantly many heads with multiple single-head attention layers. [**?**]

Algorithm 1 Causal Self-Attention, ATTN

Require: Parameter $\theta_{\mathsf{ATTN}} = (W_Q, W_K, W_V, W_O)$, Input embedding $h = (h_1, \ldots, h_a) \in \mathbb{R}^d \times \cdots \times \mathbb{R}^d$.

Ensure: Output embedding $h' = (h'_1, \ldots, h'_a) := \mathsf{ATTN}_{\theta_{\mathsf{ATTN}}}(h_1, \ldots, h_a)$.

1: $q_i := W_Q h_i,\ k_i := W_K h_i,\ v_i := W_V h_i,\ \forall i \in [1, \ldots, a]$
2: $s_i := \text{softmax}(\langle q_i, k_1\rangle, \ldots, \langle q_i, k_i\rangle) \| (0, \ldots, 0)$
3: $h'_i := W_O \sum_{j=1}^{a} (s_i)_j v_j$

Feed Forward Network: It's a mapping from $\mathbb{R}^d$ to $\mathbb{R}^d$. Given $\theta_{\mathsf{FF}} = (W_1, b_1, W_2, b_2) \in \mathbb{R}^{d\times d} \times \mathbb{R}^d \times \mathbb{R}^{d\times d} \times \mathbb{R}^d$, it's defined as $\mathsf{FF}_{\theta_{\mathsf{FF}}}(h) := W_2\ \text{relu}(W_1 h + b1) + b2$, where $\text{relu} : \mathbb{R}^d \to \mathbb{R}^d$ replaces each position x_i by $\max(0, x_i)$.

Output Layer: It's a mapping from $\mathbb{R}^d$ to a probability distribution over Σ. Given $\theta_{\mathsf{OUTPUT}} \in \mathbb{R}^{|\Sigma|\times d}$, it's defined as $\mathsf{OUTPUT}_{\theta_{\mathsf{OUTPUT}}}(h) := \text{softmax}(\theta_{\mathsf{OUTPUT}} h)$.

All these parts come together for the distribution generator process in the algorithm 2.

The OUTPUT layer and the ATTN layer uses a softmax function. It is defined from $\mathbb{R}^n$ to $\mathbb{R}^n$ for all n as $\text{softmax}(x)_i = exp(x_i)/\sum_{j=1}^{n} exp(x_j)$.

The second component of a transformer is the **token selector**. This algorithm consumes the probability distribution computed by the distribution generator and returns a token.

Lets $p : \Sigma \to [0, 1]$ be the output of the distribution generator. In this work we will study two selectors. The first one always returns the token with the largest probability (argmax over p, where p is the output of the distribution generator). The other, works non-deterministically. It returns the token $\sigma \in \Sigma$ with probability $p(\sigma)$.

Definition 2.1.1 (Deterministic transformer)**.** We call a transformer deterministic when the token selector is the one taking argmax over the distribution. We say that a family of

Algorithm 2 Distribution generator

Require: Transformer parameter $\theta = (\theta_{\mathsf{TE}}, \theta_{\mathsf{PE}}, \{\theta^l_{\mathsf{ATTN}}, \theta^l_{\mathsf{FF}}\}_{l=0}^{L-1}, \theta_{\mathsf{OUTPUT}})$ and tokens of the tape $\alpha \in \Sigma^*$.

Ensure: Output distribution $p_\theta(\cdot|\alpha)$.

1: $h_i^{(0)} \leftarrow \theta_{\mathsf{TE}}(\alpha_i) + \theta_{\mathsf{PE}}(i),\ \forall 1 \leq i \leq |\alpha|$
2: **for** $l = 0, \ldots, L-1$ **do**
3: $\quad (h_1^{(l+0.5)}, \ldots, h_{|\alpha|}^{(l+0.5)}) \leftarrow (h_1^{(l)}, \ldots, h_{|\alpha|}^{(l)}) + \mathsf{ATTN}_{\theta^{(l)}_{\mathsf{ATTN}}}(h_1^{(l)}, \ldots, h_{|\alpha|}^{(l)})$
4: $\quad h_i^{(l+1)} \leftarrow h_i^{(l+0.5)} + \mathsf{FF}_{\theta^{(l)}_{\mathsf{FF}}}(h_i^{(l+0.5)}),\ \forall 1 \leq i \leq |\alpha|$
5: **end for**
6: $p_\theta(\cdot|\alpha) \leftarrow \mathsf{OUTPUT}_{\theta_{\mathsf{OUTPUT}}}(h_{|\alpha|}^{(L)})$

deterministic transformers $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$ recognize a language $\mathcal{L}$ when:

$$\alpha \in \mathcal{L} \iff \mathsf{TF}^*_{|\alpha|}(\alpha) = q_{accept}$$

As mentioned in chapter 1, there is a theoretical gap in the area. está bien dicho theoretical gap? A study of non-determinism in transformers is missing.

When TF is a deterministic transformer, $\mathsf{TF}^i(\alpha)$ is an exact symbol for all $\alpha \in \Sigma^*$ and $i \in \mathbb{N}$. In counterpart, when the token selection process of the transformer samples the distribution, it becomes a random variable. Therefore, we introduce the following definition.

Definition 2.1.2 (Probabilistic transformer)**.** We call probabilistic transformers to the transformers that sample the distribution for selecting the next token. We say that a family of probabilistic transformers $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$ recognize a language $\mathcal{L}$ when:

$$\alpha \in \mathcal{L} \iff \Pr[\mathsf{TF}^*_{|\alpha|}(\alpha) = q_{accept}] > 2/3$$

2.2 Finite precision modeling

I'm missing good symbols for ∞ y $-\infty$.

In practice, training and inference of transformers are typically done with 16- or 32-bit floating point numbers and there is active work in reducing that number. That's why in this thesis we follow the decision of [**?**] to work on constant-precision transformers, where the output of each arithmetic operation is rounded to the closest floating point number representable by a fixed number of digits following IEEE 754 standard. This analysis avoids the unrealistic infinite precision assumption made by prior works [**?**]. The definitions stated in the section are inherited from [**?**].

Below we give a formal definition of the floating-point number and rounding operation. We use $\phi(a) = \sum_{i=1}^{k} 2^{k-i} a_i$ to denote the value of binary number represented by $a \in \{0,1\}^k$ for any $k \in \mathbb{N}^+$.

Definition 2.2.1 (Floating-point Representation)**.** Let e be the number of bits for exponents and s be the number of bits for significand. A $(e + 2s + 1)$-bit binary string $a = (a_1, a_2, \ldots a_{e+2s+1}) \in \{0,1\}^{e+2s+1}$ is a floating-point binary representation of number $\phi_{e,s}(a) \triangleq \text{sign}(a) \cdot 2^{\text{exponent}(a)} \cdot \text{significand}(a)$ with e-bit exponent and $2s$-precision, where the sign is $\text{sign}(a) \triangleq 2a_1 - 1$, the significand is $\text{significand}(a) \triangleq 2^{-s}\phi(a_{2:2s+1})$, and the

exponent is $\text{exponent}(a) \triangleq \phi(a_{2s+2:2s+e+1}) - 2^{\max(0,e-1)}$. We define the set of numbers expressible with e bits of exponent and s bits of significand as $\mathbb{F}_{e,s}$. We define $\infty_{e,s}$ as the maximum number in $\mathbb{F}_{e,s}$. Similarly, we define $-\infty_{e,s}$ as the minimum. Since in this work we only analyze constant precision transformers, we will omit the subscripts of ∞ and $-\infty$.

Definition 2.2.2 (Correct Rounding)**.** For any $x \in \mathbb{R}$, we define correct rounding $\text{round}(x, \mathbb{F}_{e,s})$ as the closest number to x in $\mathbb{F}_{e,s}$. We break the tie by picking the one with a smaller absolute value. In particular, we denote the rounding operation with e-bit exponent, $2s$-bit precision by $\text{round}_{e,s}(\cdot) \triangleq \text{round}(\cdot, \mathbb{F}_{e,s})$, which is also denoted by $[\cdot]_{e,s}$ for convenience. We extend the definition of round and $\text{round}_{e,s}$ to vector inputs by rounding coordinate-wisely.

Our notion of floating-point number simplifies the IEEE 754 Standard for Floating-point Arithmetic [**?**] by removing ∞ and $-\infty$. When overflow happens, we always round the output to ∞, and when underflow happens to $-\infty$. For unary functions like $\exp(\cdot)$ and binary functions including addition, subtraction, multiplication, and division, we simply define their rounded version by rounding their outputs. Whenever division by 0 happens, we define the output as 0.[1]

Next, we define finite-precision summation over more two numbers by decomposing it as a chain of rounded binary addition in a fixed order. [2]

Definition 2.2.3 (Summation with Iterative Rounding)**.** For any $s, n \in \mathbb{N}^+$ and vector $x \in \mathbb{R}^n$, we define summation with iterative rounding to e bit exponent and $2s$-bit precision as $\mathsf{sum}_{e,s} : \cup_{n \in \mathbb{N}^+} (\mathbb{F}_{e,s})^n \to \mathbb{F}_{e,s}$, where for any $n \in \mathbb{N}^+$ and $x \in \mathbb{R}^n$,

$$\mathsf{sum}_{e,s}(x) \triangleq \Bigg[\Bigg[\Big[[x_1 + x_2]_{e,s} + x_3 \Big]_{e,s} + \cdots x_{n-1} \Bigg]_{e,s} + x_n \Bigg]_{e,s} .$$

We further define the following operations:

- Finite-precision inner product: $\langle x, y \rangle_{e,s} \triangleq \mathsf{sum}_{e,s}(x \odot y)$;
- Finite-precision matrix product: $(A \times_{e,s} B)_{i,j} \triangleq \left\langle (A_{i,:})^\top, B_{:,j} \right\rangle_{e,s}$;
- Finite-precision softmax: $\text{softmax}_{e,s}(x) \triangleq \Big[[\exp(x)]_{e,s} / \mathsf{sum}_{e,s}([\exp(x)]_{e,s}) \Big]_{e,s}$.

From the saturation and the iterative rounding presented by [**?**], some properties appear that are important to remark:

- $exp(-\infty) = 0$
- $exp(\infty) = \infty$
- $-\infty \cdot \infty = -\infty$
- $\infty \cdot \infty = \infty$

[1] Notice that the only division is in the softmax. If it's the one in the attention mechanism, it's reasonable to give an attention score of 0 to every vector. If it's the one in the output layer it means that all symbols have an extremely small probability, therefore is reasonable to assign the same probability to all of them.

[2] Technically speaking, instead of a chain, the summation could also proceed like a tree.

- $\forall x \in \mathbb{F}_{e,s}$ it holds that $x + 0 = x$
- $\forall x \in \mathbb{F}_{e,s}$ it holds that $x - x = 0$
- $\forall x \in \mathbb{F}_{e,s}$ it holds that $x + \infty + \infty + -\infty = 0$

2.3 Transformer Complexity classes

Maybe rethink this title?

As discussed in the first chapter, there are different resources to bound in the transformers to limit their expressive power. In this thesis, we will analyze the effect of bounding the number of steps of chain of thought that the transformers are allowed to make before answering. With that in mind, we define the following complexity classes.

Given two functions $T(n)$ and $d(n)$ we define the complexity class $\mathsf{CoT}[T(n), d(n)]$. Informally, a language $\mathcal{L}$ is in $\mathsf{CoT}[T(n), d(n)]$ if and only if there is a deterministic family of transformers with constant depth, embedding size $d(n)$ and budget $T(n)$ that recognizes it.

Definition 2.3.1 ($\mathsf{CoT}[T(n), d(n)]$)**.** A language $\mathcal{L}$ is in $\mathsf{CoT}[T(n), d(n)]$ if and only if there is an integer L and two functions $T'(n) = O(T(n)), d'(n) = O(d(n))$, such that for every positive integer n, there is an L-layer deterministic transformer, denoted by TF_n with embedding size $d'(n)$, that can output $\mathcal{L}(\alpha)$ given any input α in Σ^n, using $T'(n)$ steps of chain of thought.

In the same way, we define the classes of languages recognized by probabilistic transformers. Given two functions $T(n)$ and $d(n)$ we define the complexity class $\mathsf{RCoT}[T(n), d(n)]$. Informally, a language $\mathcal{L}$ is in $\mathsf{RCoT}[T(n), d(n)]$ if and only if there is a probabilistic family of transformers with constant depth, embedding size $d(n)$ and budget $T(n)$ that recognizes it.

Definition 2.3.2 ($\mathsf{RCoT}[T(n), d(n)]$)**.** A language $\mathcal{L}$ is in $\mathsf{RCoT}[T(n), d(n)]$ if and only if there is an integer L and two functions $T'(n) = O(T(n)), d'(n) = O(d(n))$, such that for every positive integer n, there is an L-layer probabilistic transformer, denoted by TF_n with embedding size $d'(n)$, that can output $\mathcal{L}(\alpha)$ given any input α in Σ^n, using $T'(n)$ steps of chain of thought with probability higher than $2/3$.

2.4 Circuit Complexity

2.5 Known results

3. GENERAL PROPERTIES

In order to better understand $\mathsf{CoT}[T(n), d(n)]$ classes, it was needed to fill a gap in the literature related to the properties of these classes. For that, in this section we introduce a notion of normalization for transformers and prove that $\mathsf{CoT}[T(n), d(n)]$ is closed under boolean operations. As middle step, it was needed to build certain primitives operations which are also explained in this chapter.

3.1 Transformers Primitives

Flagging Tokens and Positions

We will see later on that we will want to be able to identify when a certain token appeared in the tape. Also, we would like to know when a certain position is reached or if a certain vector corresponds to a specific position. This last task is not naturally done because of the extreme parallelism in the model.

For flagging a token we use the token embedding, and in the same way, for flagging a position we use the position encoding. Either way, the strategy is exactly the same.

For each flag that we are looking to add, we will extend the dimension embedding by 1. Notice that we can add a constant number of flags to a family of transformers, and it will remain in the same $\mathsf{CoT}[T(n), d(n)]$ class.

The idea for flagging is straight forward. We will have an extra coordinate that will work as the flag. This coordinate will have two possible values, to be defined. It will have the on value f_{on} if the vector is flagged and the off value f_{off}, in the other case.

For example, let's be TF a transformer and θ_{TE} its token embedding function. We will define a transformer TF' with the same behavior, but it will have flagged every vector that came from the token σ_f.

To have less notation noise, we will add the flag in the last coordinate of the embedding, but this can be done in any position.

Let's define each component of TF' based on the components of TF. First, let's see the token embedding function. It will be the same except that in the new coordinate it will put a f_{on} if the token that is consuming is the one to be flagged and f_{off} otherwise.

$$(\theta'_{TE}(\sigma))_j = \begin{cases} (\theta_{TE}(\sigma))_j & \text{if } j \neq d+1 \\ f_{\mathsf{on}} & \text{if } j = d+1 \text{ and } \sigma = \sigma_f \\ f_{\mathsf{off}} & \text{if } j = d+1 \text{ and } \sigma \neq \sigma_f \end{cases}$$

For the position encoding is enough to always put a 0 in the new coordinate. Then is only necessary to extend the matrix of the other layers to the new dimension. This can be done with a new row or column with zeros. The behavior of TF' will be the same as TF.

If we are looking to add more coordinates to the embedding, it can be done adding a flag that has the same value for f_{on} and f_{off}.

Preserve and ignore coordinates through ATTN and FF

When defining new transformers over existing ones we will want to extend their embedding dimension. We would like to preserve the behavior of the old transformer in the existing coordinates but ignore the new ones. Let's see how to extend the dimensions of the ATTN and FF layers in a way that we accomplish this.

If we want to ignore the ith coordinate through a layer of attention, it's enough to put zeros in the ith column and row of W_Q, W_K, W_V, W_O. This can be done with as many coordinates as desired. Note that because the ith row of W_O is zero, the output vector of the ATTN layer will have a zero in the ith coordinate. This is the point that makes it possible to not just ignore the value of the ith coordinate, but to not affect it. This happens because the result of the attention is added to the original vector.

If we are looking to only preserve but not ignore a coordinate during an ATTN layer, it's enough to have zeros in only its corresponding row of W_O.

The same applies in the feed forward layer, putting zeros in the ith column and row of the matrix and vectors of the FF makes it ignore the value of the ith coordinate. Also in the result of the FF will appear a zero in the ith coordinate.

Add vector and constant functions

Let's define a mechanism for adding one specific vector to a flagged one. This technique works even when there are more than one flagged vector.

The flag used for the vectors should have $f_{\mathsf{on}} = 1$ and $f_{\mathsf{off}} = 0$. We will assume the flag is in the last coordinate.

Let's call v the vector that we want to add. We will do it with one FF layer. We will construct a layer such that $FF(h) = \mathbb{I}(h \text{ is flagged}) * v$.

$$W_1 = id \qquad W_2 = \begin{pmatrix} 0 & \dots & 0 & v_1 \\ \vdots & \ddots & \vdots & \vdots \\ 0 & \dots & 0 & v_d \end{pmatrix} \qquad b_1 = b_2 = \begin{pmatrix} 0 \\ \vdots \\ 0 \end{pmatrix}$$

Notice that if the coordinate for the flag had been the kth instead of the last one, then v would have appeared in the kth column of W_2 instead of the last.

An interesting use of this technique is that allow us to transform any vector in whatever we want, i.e. we can apply constant functions. This is done exploiting the finite representation system. We can saturate the vector to the ∞ and then make it 0 to finally add our objective.

$$h \leftarrow \left(\left(\left(h + \begin{pmatrix} \infty \\ \vdots \\ \infty \end{pmatrix}\right) + \begin{pmatrix} \infty \\ \vdots \\ \infty \end{pmatrix}\right) + \begin{pmatrix} -\infty \\ \vdots \\ -\infty \end{pmatrix}\right) + v$$

Make a vector ignore others in an ATTN layer

We will find very useful to be able for some vectors to ignore others in the ATTN layers. For a vector to ignore other, we will need for the attention score to be $-\infty$ so the softmax make it zero.

Let's say we want for the lth vector to ignore a set of vectors M. Therefore, what we are looking for is for $\langle q_l, k_m \rangle = -\infty$ when $m \in M$, but we don't want to modify the attention score of others pairs. We will use the saturation error for getting to our objective, since $\langle q_l, k_m \rangle = \sum_{x=1}^{d} (q_l)_x (k_m)_x$ if we make for the last two terms to be $-\infty$ when $m \in M$, we will get what we are looking for. Since we don't want to affect other vectors, every other pair should have 0 as the last to terms.

We will make $(k_i)_{d-1}$ and $(k_i)_d$ to be $-\infty$ when $i \in M$, and $(q_i)_{d-1}$ and $(q_i)_d$ to be ∞ when $i = l$ and 0 otherwise. With these values the only attention scores modified are the ones of the lth vector since every other will have 0 in the last two coordinates of the query thus making the last two terms of the internal product 0 and therefore not affecting the score.

For getting those values in the keys and queries vectors we can just flag them and adjust the query and key matrix to have what we want in those coordinates.

Linear transformations

Although it looks that transformers are a model that should be able to perform linear transformations in a native way, it is not the case. Applying a linear transformation to a vector is not direct since in every step of ATTN and FF the result of the layer is added to the original vector instead of replacing it. A sharp reader maybe would think that this can be solved applying the transformation $M - I$ when we want to transform the vector x to Mx, since $x + (M - I)x = x + Mx - Ix = Mx$. This does not hold in our model since the addition and multiplication in the finite representation of the numbers used is not commutative. For avoiding the representation error, we will extend the embedding dimension to get more memory per vector such that we can store in new coordinates the value of Mx and then replace it in the original ones. After applying this process we will have corrupted all vectors except the one we are interested in.

The strategy we will use is the following. For having a clearer notation, we will say that the vector to which we are applying the linear transformation is the fth and the matrix of the linear transformation is M. We will do use two consecutive ATTN layers where the fth vector will only attend to itself. We can achieve this making it ignore all the others vectors except itself, then its attention score to itself will be 1.

The first layer will compute the result of the transformation and store it in new coordinates added to the embedding to be used as memory. The second layer will move the result to the right place.

For doing this we will need to extend the embedding dimension with enough coordinates to store the result of the transformation. Therefore, if we are working with a transformer of embedding dimension d and want to do a linear transformation of an entire vector, we will need to extend the dimension by a factor of 2. The first d coordinates will be for the original vector and the second d coordinates will be the ones using as memory. We will assume that the d coordinates used as memory are set to 0 by the layers previous to this mechanism. This can be achieved maintaining them as 0 since the beginning or transformed to 0 with the primitive described in the previous section.

$$h_f = \begin{pmatrix} x_1 \\ \vdots \\ x_d \\ 0 \\ \vdots \\ 0 \end{pmatrix} \xrightarrow{\text{first ATTN layer}} h_f = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ (Mx)_1 \\ \vdots \\ (Mx)_d \end{pmatrix} \xrightarrow{\text{second ATTN layer}} h_f = \begin{pmatrix} (Mx)_1 \\ \vdots \\ (Mx)_d \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

this should be an epigraph:a graphic representation of the technique

Let's construct each of the ATTN layers. As said before, in both layers we will need for h_f to only attend to itself. Therefore, we define W_Q ans W_K using the strategy of the previous section.

Let's remember from the definition of the ATTN mechanism (1) that the output of the ATTN layer is $h'_i = W_O \sum_{j=1}^{a} (s_i)_j v_j$. Let's analyze what h'_f looks by now:

$$h'_f = W_O \sum_{j=1}^{a} (s_f)_j v_j = W_O v_f = W_O(W_V h_f)$$

Then, defining W_O and W_V as follows

$$W_V = id \qquad\qquad W_O = \begin{pmatrix} -id^{\mathbb{R}^{d\times d}} & 0^{\mathbb{R}^{d\times d}} \\ M & 0^{\mathbb{R}^{d\times d}} \end{pmatrix}$$

makes us get $h'_f = (-x_1, \ldots, -x_d, (Mx)_1, \ldots, (Mx)_d, 1)$, which added to h_f gives us what we were looking for after the first layer.

For the second ATTN layer, we can use the same mechanism for making h_f only attend to itself. And defining W_V and W_O as follows

$$W_V = id \qquad\qquad W_O = \begin{pmatrix} 0^{\mathbb{R}^{d\times d}} & id^{\mathbb{R}^{d\times d}} \\ 0^{\mathbb{R}^{d\times d}} & -id^{\mathbb{R}^{d\times d}} \end{pmatrix}$$

we get $h'_f = ((Mx)_1, \ldots, -(Mx)_d, -(Mx)_1, \ldots, -(Mx)_d)$, which added to h_f gives us what we were looking for after the second layer.

Max coordinate

We will see later on that we will need to be able to take the maximum of a set of coordinates. The operation we are looking to do is the following:

$$\begin{pmatrix} a \\ b \end{pmatrix} \to \begin{cases} \begin{pmatrix} a \\ 0 \end{pmatrix} & \text{if } a > b \\ \\ \begin{pmatrix} 0 \\ b \end{pmatrix} & \text{if } b > a \end{cases}$$

We will define it for taking the maximum between only two coordinates, but it can be extended to a larger set repeating the operation as a tournament.

This operation can be done with the following primitives:

$$\begin{pmatrix} a \\ b \\ 0 \\ 0 \\ \vdots \end{pmatrix} \xrightarrow{\text{FF layer}} \begin{pmatrix} a \\ b \\ \text{relu}(a-b) \\ \text{relu}(b-a) \\ \vdots \end{pmatrix} \xrightarrow[\text{the coordinates with the relu}]{\text{Multiply twice by } \infty} \begin{pmatrix} a \\ b \\ \infty \cdot \mathbb{I}(a>b) \\ \infty \cdot \mathbb{I}(b>a) \\ \vdots \end{pmatrix} \longrightarrow$$

$$\xrightarrow[\substack{\text{coordinates with} \\ \text{the comparison}}]{\text{Divide by } \infty \text{ the}} \begin{pmatrix} a \\ b \\ \mathbb{I}(a>b) \\ \mathbb{I}(b>a) \\ \vdots \end{pmatrix} \xrightarrow[\text{operation}]{\text{Special}} \begin{pmatrix} a \cdot \mathbb{I}(a>b) \\ b \cdot \mathbb{I}(b>a) \\ 0 \\ 0 \\ \vdots \end{pmatrix} = \begin{cases} \begin{pmatrix} a \\ 0 \\ \vdots \end{pmatrix} & \text{if } a > b \\ \\ \begin{pmatrix} 0 \\ b \\ \vdots \end{pmatrix} & \text{if } b > a \end{cases}$$

Implementation of the special operation

The operation we are looking for to do it's not a linear transformation, therefore we need to define a new technique. Remember that one of $\mathbb{I}_a$ and $\mathbb{I}_b$ is a 1 and the other is a 0. Notice that $\mathbb{I}_b = 1 - \mathbb{I}_a$ and $\mathbb{I}_a = 1 - \mathbb{I}_b$.

$$\begin{pmatrix} a \\ b \\ \mathbb{I}_a \\ \mathbb{I}_b \end{pmatrix} \xrightarrow[\text{operation}]{\text{Special}} \begin{pmatrix} a \cdot \mathbb{I}_a \\ b \cdot \mathbb{I}_b \\ 0 \\ 0 \end{pmatrix}$$

This operation can be performed with three layers of FF. What we will do is the following:

$$\begin{pmatrix} a \\ b \\ \mathbb{I}_a \\ \mathbb{I}_b \end{pmatrix} \xrightarrow[\text{operation}]{\text{Special}} \begin{pmatrix} a & + & \infty \cdot \mathbb{I}_b & + & \infty \cdot \mathbb{I}_b & + & -\infty \cdot \mathbb{I}_b \\ b & + & \infty \cdot \mathbb{I}_a & + & \infty \cdot \mathbb{I}_a & + & -\infty \cdot \mathbb{I}_a \\ \mathbb{I}_a & + & 0 & + & 0 & + & -\mathbb{I}_a \\ \mathbb{I}_b & + & 0 & + & 0 & + & -\mathbb{I}_b \end{pmatrix}$$

If $\mathbb{I}_a = 1$, then $\mathbb{I}_b = 0$, so the first coordinate isn't modified and the second is changed to 0 because of the finite representation properties. In the same way, if $\mathbb{I}_a = 0$, then $\mathbb{I}_b = 1$, so the second coordinate isn't modified and the first is changed to 0.

As mentioned before, this operation is performed with three FF layers. The first two will be:

$$W_1 = id \qquad W_2 = \begin{pmatrix} 0 & 0 & 0 & \infty \\ 0 & 0 & \infty & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \qquad b_1 = b_2 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

Notice that the output of this FF layer when inputted with $(a, b, \mathbb{I}_a, \mathbb{I}_b)$ will be $(\infty \cdot \mathbb{I}_b, \infty \cdot \mathbb{I}_a, 0, 0)$.

The third FF layer will be:

$$W_1 = id \qquad W_2 = \begin{pmatrix} 0 & 0 & 0 & -\infty \\ 0 & 0 & -\infty & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} \qquad b_1 = b_2 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

which inputted with $(x, y, \mathbb{I}_a, \mathbb{I}_b)$, it will output $(-\infty \cdot \mathbb{I}_b, -\infty \cdot \mathbb{I}_a, -\mathbb{I}_a, -\mathbb{I}_b)$

Repeat a token once it's printed

There are going to be constructions were we would like to be able to, once the transformer prints a token, repeat it indefinitely. We can achieve this easily, making use of some of the tools that we had defined. First, we need to normalize the transformer. i know i define normalization later on, but i think this is the place in the text where i should write this primitive. Second, we need to flag the token with the TE. Third, we will apply a constant function to the flagged vector. Since we are trying to replicate a token that was printed, i.e. not part of the input, its vector will be the last one. This is because once the transformer prints it for the first time, we will start forcing it to repeat it. It's important that the constant function is applied in the last layers so that the vector that goes into the OUTPUT layer (which is just a projection) is the one making all the probability going to the desired token.

Notice that this only works if the token does not appear in the input word. That case can be solved using the non-uniformity of the model. If the input word is of size n, with the PE we can flag the first n vectors. Then, we can use that flag to make them 0 in the coordinate used by the TE for flagging the token to repeat. We make them 0 using a constant function as described in a previous section.

Force to halt after certain number of steps of CoT

With a similar strategy to the previous section, with the PE we can flag when a certain position is reached. Then, if the transformer is normalized, we can make the last vector what ever we want (since it will be the flagged by the PE). Particularly, we can force it to concenter the probability in q_{accept} or q_{reject}.

Notice that this primitive with the one before, makes us able to always assume that the transformer uses all of its budget before halting. If that wasn't the case, we can extend the alphabet with two dummy versions of the decision tokens and force the transformer to print those tokens instead of the real ones[1]. Then, we can force it to repeat the one it printed until the last position of the budget. At that point, we can force it to print the real decision token corresponding to the one it was repeating (if it already printed one) or force it to choose one.

[1] This is achieved giving the probability of each decision token to its dummy version.

Copy one vector into another with ATTN

We will see that a useful operation will be to move values from a vector v_i to a vector v_j. This can only be done if $i < j$ since the attention mechanism does not allow for a vector to attend to other that is to the right, i.e, v_k can attend to v_l iff $k \leq l$.

We will copy values from v_i to v_j in two steps. First, we will make 0 the coordinates of v_j that will receive the values. Second, we will make v_j only attend to v_i in an attention layer. In said attention layer, we will have a $W_V = id$ and W_O such that it outputs the desire coordinates of v_i in the coordinates we want to copy them. For example if we were looking for copying the kth coordinate of v_i to the lth coordinate of v_j, W_O will have 0 en every coordinate and 1 in the row l, column k. Then the vector coming out of the ATTN layer will have 0 in every coordinate, except in the lth, where it will be $(v_i)_k$.

3.2 Normalization

For an easier handling of transformers, we define a notion of normalized transformer. This notion will specify properties of the OUTPUT matrix and of the vector consumed by this step.

Definition 3.2.1 (Normalized transformer)**.** We will say a transformer is normalized when it meets two conditions. First, the matrix of the OUTPUT has to be a projection, i.e. it's a diagonal matrix with only ones in its diagonal. Second, the last vector after the last iteration has to only be composed of $-\infty$ in every projected coordinate except, one which has to be ∞. The non projected coordinates are free to be any value.

The motivation for this definition is to simplify the OUTPUT layer (first constraint) and to have more control over the probability distribution generated (second constraint). The exact values of the last vectors are such that after projecting and applying softmax in the OUTPUT layer we get a deterministic distribution with 100% of the probability concentered in only one token.

Algorithm for normalizing

not sure about title

Let's see how to normalize any transformer. Let's take a transformer and modify it to meet the first constraint.

Every linear transformation $M : \mathbb{R}^d \to \mathbb{R}^{|\Sigma|}$ with $d \geq |\Sigma|$ can be decomposed in two steps. [2]

$$\begin{pmatrix} x_1 \\ \vdots \\ x_d \end{pmatrix} \xrightarrow{M'} \begin{pmatrix} (Mx)_1 \\ \vdots \\ (Mx)_{|\Sigma|} \\ v \end{pmatrix} \xrightarrow{projection} \begin{pmatrix} (Mx)_1 \\ \vdots \\ (Mx)_{|\Sigma|} \end{pmatrix}$$

where $v \in \mathbb{R}^{d-|\Sigma|}$ can have any values.

Let's TF be a transformer and M be the matrix of the OUTPUT layer. Using the decomposition just presented, and the strategy defined in the previous section for applying

[2] We can always add more coordinates to the embedding if $d < |\Sigma|$. Since the language is fixed, it will be a constant number of coordinates, thus not changing the class of the transformer.

linear transformations, we can change the matrix of the OUTPUT layer to be only a projection. The linear transformation has to be applied to the last vector.

Let's now make it meet the second condition.

Once TF has been modified to meet the first condition, we can use the maximum operation described in the previous section to make all the positions of the vector 0 except the one that was the maximum before. Now, if the maximum is positive, multiplying by ∞, then subtracting 1 to every coordinate and multiplying by ∞ again gets us what we wanted.

$$\begin{pmatrix} 0 \\ \vdots \\ 0 \\ x_{max} \\ 0 \\ \vdots \\ 0 \end{pmatrix} \xrightarrow[\infty]{\text{multiplying by}} \begin{pmatrix} 0 \\ \vdots \\ 0 \\ \infty \\ 0 \\ \vdots \\ 0 \end{pmatrix} \xrightarrow{\text{substracting 1}} \begin{pmatrix} -1 \\ \vdots \\ -1 \\ \infty - 1 \\ -1 \\ \vdots \\ -1 \end{pmatrix} \xrightarrow[\infty]{\text{multiplying by}} \begin{pmatrix} -\infty \\ \vdots \\ -\infty \\ \infty \\ -\infty \\ \vdots \\ -\infty \end{pmatrix}$$

It could be the case that the maximum was negative, in that case the resulting vector won't be what we are looking for. Let's see how to make the maximum strictly positive so we can add this mechanism in the middle.

By this point our vector looks like $(0, \ldots, 0, x_{max}, 0, \ldots, 0)$, we are going to transform it to $(0, \ldots, 0, |x_{max}|, 0, \ldots, 0)$. Remember that the coordinates we are interested in are only the first $|\Sigma|$. Let's call v to the block containing these coordinates. Our algorithm will do the following:

$$\begin{pmatrix} v \\ \vdots \end{pmatrix} \to \begin{pmatrix} v \\ -v \\ \vdots \end{pmatrix} \to \begin{pmatrix} \text{relu}(v) \\ \text{relu}(-v) \\ \vdots \end{pmatrix} \to \begin{pmatrix} \text{relu}(v) + \text{relu}(-v) \\ \vdots \end{pmatrix}$$

where relu is applied position wise. This works since for all x it holds that $\text{relu}(x) + \text{relu}(-x) = |x|$.

The first step can be done in the same way as the first half of the linear transformations. For the second step we can take max coordinate in v and $-v$. This works because all the coordinates are 0 except one, which if it is negative will become 0 and if it is non-negative will remain the same. For adding both vectors we can use one FF with the following parameters. Notice that this works since all the coordinates of $\text{relu}(-v)$ are non-negative.

$$W_1 = id^{d \times d} \qquad W_2 = \begin{pmatrix} 0^{|\Sigma| \times |\Sigma|} & id^{|\Sigma| \times |\Sigma|} & 0^{|\Sigma| \times (d-2|\Sigma|)} \\ 0^{d-|\Sigma| \times |\Sigma|} & 0^{d-|\Sigma| \times |\Sigma|} & 0^{d-|\Sigma| \times d-2|\Sigma|} \end{pmatrix} \qquad b_1 = b_2 = \left(0^{d \times d}\right)$$

3.3 Boolean Operations

The main objective of this chapter was to prove that the $\mathsf{CoT}[T(n), d(n)]$ classes are closed under boolean operations.

Let's show that the complement of a language in $\mathsf{CoT}[T(n), d(n)]$ is in $\mathsf{CoT}[T(n), d(n)]$ and that the disjunction of two language of the same class, stays in the same class.

Complement of a language

Let's $\mathcal{L} \in \mathsf{CoT}[T(n), d(n)]$, then there is a family of transformers $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$ of embedding size $d(n)$ that after $T(n)$ steps of CoT prints q_{accept} or q_{reject} depending on if the input word is in the language. For each of the transformers in the family we can define one that behaves exactly the same (internally and externally) but prints the flipped token.

Let's TF be a transformer, we will construct TF' that flips the decision token of TF, i.e if $\mathsf{TF}^*(\alpha) = q_{accept}$, then $\mathsf{TF}'^*(\alpha) = q_{reject}$ and if $\mathsf{TF}^*(\alpha) = q_{reject}$, then $\mathsf{TF}'^*(\alpha) = q_{accept}$.

This can be easily done swapping the rows associated to q_{accept} and q_{reject} in the matrix of the OUTPUT layer of TF. Thus, when the largest probability is associated to q_{accept} in TF, that same number will be associated to q_{reject} in TF.

Formally, TF' will be exactly the same as TF except in the OUTPUT layer, where the matrix will be different. If M is the matrix of the OUTPUT layer of TF, i is the position of the token q_{accept} and j the one of q_{reject}, then the matrix of the OUTPUT layer of TF will be $M' = PM$ where P is the matrix that flips the rows i and j.

$$M'_{kl} = \begin{cases} 1 & \text{if } k = l,\ k \neq i \text{ and } k \neq j \\ 1 & \text{if } k = j \text{ and } l = i \\ 1 & \text{if } k = i \text{ and } l = j \\ 0 & \text{otherwise} \end{cases}$$

TF' has the same budget and embedding size that TF.

We can define the family $\{\mathsf{TF}'_n\}_{n\in\mathbb{N}}$ where TF'_i is the negation of TF_i. Then $\{\mathsf{TF}'_n\}_{n\in\mathbb{N}}$ computes the complement of $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$. Since for every $i \in \mathbb{N}$, TF'_i has the same budget and embedding size that TF_i, $\bar{\mathcal{L}}$ is in $\mathsf{CoT}[T(n), d(n)]$.

Disjunction of two languages

Let's $\mathcal{L}_1, \mathcal{L}_2 \in \mathsf{CoT}[T(n), d(n)]$, then there are is a family of transformers $\{\mathsf{TF1}_n\}_{n\in\mathbb{N}}$ (resp. $\{\mathsf{TF2}_n\}_{n\in\mathbb{N}}$) of embedding size $d_1(n)$ (resp. $d_2(n)$) $\in O(d(n))$ that after $T_1(n)$ (resp. $T_2(n)$) $\in T(n)$ steps of CoT that computes $\mathcal{L}_1$ (resp. $\mathcal{L}_2$).

We will construct a family $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$ that will compute $\mathcal{L}_1 \cup \mathcal{L}_2$. It will do it in $T_3(n) = T_1(n) + T_2(n) + 1 \in O(T(n))$ steps of CoT and it will have embedding size $d_3(n) \in O(d(n))$.

The idea behind the construction will be to run first one of the transformers, then run the other and finally compare both results. Let's assume that when the input is α, the tape of the transformer $\mathsf{TF1}$ ends up looking like $\alpha\sigma$q1, where $\sigma \in \Sigma^*$ and q1 is a decision symbol of $\mathsf{TF1}$. In the same way, let's call $\alpha\tau$q2 to the content of the tape of $\mathsf{TF2}$ when it halts after being input with α. The tape of the transformer computing the disjunction of $\mathsf{TF1}$ and $\mathsf{TF2}$, at the end of the computation will look like $\alpha\sigma$q1τq2q3, where σ and τ are the intermediate tokens generated by $\mathsf{TF1}$ and $\mathsf{TF2}$, q1 and q2 are their acceptance or rejection tokens and q3 is the disjunction of q1 and q2.

Let's construct $\mathsf{TF3}$ from $\mathsf{TF1}$ and $\mathsf{TF2}$. Since we can't change the body of the transformer during the iterations we will require to use some tricks.

We grow the dimension of the embedding for running the body of both transformers at the same time over the same vectors but independently. In the final layers we will choose which probability distribution will go to the output layer, this will make $\mathsf{TF3}$ print

the token generated by TF1 or TF2. As always, we will assume that the transformers are normalized.

We will use the first half of the embedding for the transformer TF1 and the second for the transformer TF2. We will have extra coordinates for flags at the end, but we will ignore them by now.

$$\mathsf{TE3}(\sigma)_i = \begin{cases} \mathsf{TE1}(\sigma)_i & \text{if } i \leq d_1(|\alpha|) \\ \mathsf{TE2}(\sigma)_{i-d_2(|\alpha|)} & \text{if } d_1(|\alpha|) < i \leq d_2(|\alpha|) \\ 0 & \text{if } d_2(|\alpha|) < i \end{cases}$$

$$\mathsf{PE3}(n)_i = \begin{cases} \mathsf{PE1}(n)_i & \text{if } i \leq d_1(|\alpha|) \\ \mathsf{PE2}(n)_{i-d_1(|\alpha|)} & \text{if } d_1(|\alpha|) < i \leq d_2(|\alpha|) \\ \text{flags} & \text{if } d_2(|\alpha|) < i \end{cases}$$

there is a problem in this definition, PE2 is defined over $[|\alpha| + T2(|\alpha|)]$ and here we are using it over $[|\alpha| + T2(|\alpha|) + T1(|\alpha|)]$. This is fixed later on but im not sure if i want to note it right now.

The transformer TF3 will have one layer for each layer of TF1 and each of TF2. The first layers, will correspond to the body of TF1. They will be constructed ignoring the half of the embedding where the coordinates coming from TF2 are. The second half will be the other way around but with the layers of TF2 and ignoring TF1 coordinates. Then, all the matrix in the layers of the first part of the body of TF3 will look like $W3_1$ and the ones in the second layers will look like $W3_2$ where $W1$ and $W2$ are the matrix of the corresponding layers in TF1 and TF2.

$$W3_1 = \begin{pmatrix} W1 & 0 \\ 0 & 0 \end{pmatrix} \qquad W3_2 = \begin{pmatrix} 0 & 0 \\ 0 & W2 \end{pmatrix}$$

It is important to notice that the values of the coordinates corresponding to one transformer does not affect the values of the other. Thus, if the tape of the transformer has content β, then at the end of the body of TF3, the last vector would look like (x, y, flags) where x is the last vector at the end of the body of TF1 when its tape content is β, and y the one at the end of TF2 when its tape content is β. Therefore, if we always choose to transform said vector into $(x, 0, \ldots, 0)$ then TF3 will produce the same tape as TF1. This works for generating the first $T1(|\alpha|)$ tokens which are σq1. Thus, we would like to apply this transformation only to the vectors generating them. Therefore, we need to flag them with the position embedding. Since $v_{|\alpha|+i}$ carries the distribution to output in the $i+1$th step, i.e the $|\alpha|+i+1$th token of the tape, the vectors that we need to flag will be $v_{|\alpha|}, v_{|\alpha|+1}, \ldots, v_{|\alpha|+T1(|\alpha|)-1}$.

After running $T1(|\alpha|)$ steps of this construction, the tape will look like $\alpha\sigma$q1. We want to now generate τ. Just changing what we choose before the OUTPUT layer does not work since what the construction will produce would be TF2($\alpha\sigma$q1), and we are looking for TF2(α).

Let's see how to run TF2 in this construction ignoring the tokens produced by TF1. If we are able to make those tokens invisible to TF2, we are done. We need for TF2 to not attend to them (this can be done using the primitive presented in the previous section)

and for its position encoding to ignore them, therefore we need to "shift" it. Let's see first how to fix the position encoding of TF3.

$$\mathsf{PE3}(n)_i = \begin{cases} \mathsf{PE1}(n)_i & \text{if } i \leq d_1(|\alpha|) \\ \text{shifted}\mathsf{PE2}(n)_{i-d_1(|\alpha|)} & \text{if } d_1(|\alpha|) < i \leq d_2(|\alpha|) \\ \text{flags} & \text{if } d_2(|\alpha|) < i \end{cases}$$

$$\text{shifted}\mathsf{PE2}(n) = \begin{cases} \mathsf{PE2}(n) & \text{if } n \leq |\alpha| \\ 0 & \text{if } |\alpha| < n < T1(|\alpha|) \\ \mathsf{PE2}(n - T1(|\alpha|)) & \text{if } T1(|\alpha|) < n \end{cases}$$

The first case is for the positions occupied by the input, we don't want to change them. The second case is for the positions occupied by the tokens generated by TF1, we don't care about them. The third case is for the tokens that TF2 will generate, we want to make it believe that they are in their natural positions, i.e shifted $T1(|\alpha|)$ positions left (the positions occupied by the generated for TF1). We will also need to change the flag to the vectors after the number $\alpha + T1(|\alpha|)$ to choose the distribution generated by TF2 instead of TF1.

With the construction like this we can almost produce the tape of TF1 followed by the tape of TF2 except for one token. We need to do something especial for the first token generated by TF2. The distribution corresponding to it, it's in the vector $v_{|\alpha|}$ but the last vector of the first run of TF2 is $v_{|\alpha|+T1(|\alpha|)}$, the one coming from the last token of TF1. Therefore, before choosing the distribution, we need to copy the vector. This can be done with the primitives of the previous section.

With this last fix, we are able to generate the tape of TF1 followed by the tape of TF2. We are only missing how to compute q3. This can be easily done making $v_{|\alpha|+T1(|\alpha|)+T2(\alpha)}$ only attend to itself and to $v_{|\alpha|+T1(|\alpha|)}$, i.e. to the vectors coming from q1 and q2.

We will modify the TE3 function to add a flag in the last coordinate (let's call it f_c) for the decision tokens of TF1 and TF2.

$$\mathsf{TE3}(\sigma)_{f_c} = \begin{cases} 1 & \text{if } \sigma = q_{accept} \vee \sigma = q_{accept} \\ 0 & \text{otherwise} \end{cases}$$

Then if we add $v_{|\alpha|+T1(|\alpha|)}$ to $v_{|\alpha|+T1(|\alpha|)+T2(\alpha)}$ we will have 1 or 2 in the coordinate f_c iff TF1 or TF2 accepted α. Now we can make all of $v_{|\alpha|+T1(|\alpha|)+T2(\alpha)}$ 0, except the f_c coordinate. Before going to the output layer we need to move the value of f_c to the coordinate that will project to the probability of q_{accept} and add some number larger than 0 and smaller than 1 to the coordinate that will define the probability of q_{reject}.

Therefore, the token with the maximum probability will be q_{accept} iff TF1 or TF2 accepted α, and it will be q_{reject} in the other case. Then, TF3 computes the union of the languages of TF1 and TF2. The number of steps of CoT that it needs is the same as TF1 plus TF2 plus 1 extra for taking the disjunction, thus in $O(\dot{})$ notation it takes the same as the maximum between TF1 and TF2. The embedding dimension of TF3 is the same as the dimension of TF1 plus TF2 plus some constant number of flags, thus in $O(\dot{})$ notation it's the same as the maximum between TF1 and TF2.

Therefore, the language computed by the family of transformers $\{\mathsf{TF3}_n\}_{n\in\mathbb{N}}$ is in $\mathsf{CoT}[T(n), d(n)]$. Thus, $\mathcal{L}1 \cup \mathcal{L}2 \in \mathsf{CoT}[T(n), d(n)]$.

4. MAIN RESULTS

5. CONCLUSIONS